

ESCAPE-Nav: Experience-Shaped Causally Aligned Perception–Execution for Asynchronous VLM Navigation

Anonymous Authors

Abstract—상대 PointGoal은 목표 방향을 알려주지만 벽이나 교차로의 실행 가능한 local route는 정하지 않는다. Vision–Language Model (VLM)은 multi-view RGB에서 route를 선택할 수 있으나, 느린 결과는 관측 view와 현재 state를 어긋나게 하고 실패 방향을 잇으면 deadlock과 oscillation이 반복된다. 본 논문은 delayed semantic decision을 observation-bound transaction으로 다루는 Experience-Shaped Causally Aligned Perception–Execution Navigation (ESCAPE-Nav)을 제안한다. ESCAPE-Nav는 task-conditioned body-yaw view 획득, VLM-주행 중점과 representation 별 delayed transport를 결합한다. 검증된 directed outcome은 sparse graph에 귀속되어 다음 observation 범위, candidate 순위와 recovery branch를 바꾼다. HM3D/Habitat-GS PointNav와 Unitree Go2에서 세 기전을 검증하고 task/model transfer는 보조 평가한다.

I. 서론

원격 탐사와 재난 대응 robot은 정밀한 사전 지도보다 coarse waypoint 또는 semantic object goal만을 제공받을 수 있다. 전통적인 navigation system은 occupancy/cost map의 global path를 local subgoal로 분해하지만 그 유효성은 map과 현재 robot state의 정합성에 의존한다. 반복 구조, 저특징 지형과 간헐적인 localization에서는 이 정합성이 약해지고, 이상적인 RGB-D+Global Positioning System (GPS)+Compass에서 학습된 PointNav agent도 realistic sensing과 actuation noise에서 성능이 크게 저하된다 [1]–[3]. 본 연구는 mapping이나 localization 자체를 대체하지 않고, persistent globally aligned dense map을 신뢰하기 어려울 때 coarse goal을 현재 관측에서 실행 가능한 local route로 구체화하는 문제를 다룬다.

PointNav의 relative PointGoal은 목표를 가리키는 화살표와 같지만, 벽을 피해 어느 문이나 복도를 택할지는 알려주지 않는다. 사람은 필요한 방향만 짧게 둘러보고, 막히면 시야를 넓히며, 실패한 진입과 성공한 탈출을 구분해 다음 길을 고른다. 본 연구는 이 관측–실행–기억의 원리를 monocular RGB, relative PointGoal 또는 object category와 episode-local odometry로 옮기고, 확인된 방향별 결과만 sparse experience graph에 보존한다.

Vision–language model (VLM)은 여러 방향의 영상과 목표에서 실제로 시도할 문, 복도 또는 자유 공간을 local candidate로 선택할 수 있다. 그러나 fixed forward camera는 먼저 body-yaw로 view를 얻어야 하고 VLM은 local policy보다 느리다. 관측 후 기존 trajectory를 재개해 추론과 실행을 중점하면 decision 도착 시점에는 view와 robot state가 이미 달라진다. 따라서 늦게 도착한 semantic decision이 무엇을 보고 만들 어졌고, 어느 frame에서 무엇을 뜻하며, 실제로 어떤 결과를 냈는지 끝까지 보존해야 한다. 전체 system은 Fig. 1에 제시한다.

첫 번째 난제는 *single-view partial observability*다. 벽이나 교차로에서는 어느 local branch가 목표로 이어지는지 영상으로 확인해야 하지만, 매번 모든 방향을 촬영하면 회전과 추론 비용이 커진다. 따라서 goal, motion과 failure context에 필요한 view만 선택해야 한다. 이는 active perception과 일치하지만 [4], [5], VLM navigation에서 acquisition cost와 slow inference를 실제 주행과 결합하는 문제는 충분히 다뤄지지 않았다 [6].

두 번째 난제는 *delayed semantic decision*이다. 동기 실행은 추론마다 stop-and-go motion을 만들고 단순 비동기 실행은 관측 view와 적용 state를 어긋나게 한다. 또한 world point, robot-relative metric subgoal과 image-bound pixel은 의미가 달라 오래된 pixel을 현재 영상에 복사하거나 이미 수행한 회전을 반복해서는 안 된다.

세 번째 난제는 *failure provenance*다. 막다른 장소 전체를 위험하게 표시하면 성공한 탈출 방향까지 잃지만, 최근 image/action만으로는 실패한 진입과 성공한 reverse escape의 비대칭 관계를 보존하기 어렵다. 그 결과 같은 branch의 deadlock 재진입과 $A \rightarrow B \rightarrow A$ oscillation이 반복된다.

본 논문은 Experience-Shaped Causally Aligned Perception–Execution Navigation (ESCAPE-Nav)을 제안한다. **Temporal consistency**는 decision을 observation과 적용 시점에 연결하고, **representational consistency**는 pixel · metric · world goal을 의미에 맞게 전달하며, **experiential consistency**는 owner-matched terminal outcome만 실제 directed action에 귀속한다. 이 outcome graph가 다음 observation, candidate와 recovery를 조건화하므로 ESCAPE-Nav는 *experience-shaped*이며, 이는 VLM의 online weight update가 아닌 episodic feedback을 뜻한다.

VLM은 coarse PointGoal, multi-view RGB와 place memory로 유한 후보 중 fine local goal을 선택한다. Decision은 observation identity, capture pose/time과 owner에 결합되고, world goal, capture-view pixel과 metric point는 각각 transform, viewpoint restoration/re-observation과 rebase를 거친다. Acknowledgement된 $A \rightarrow B$ 실패와 $B \rightarrow A$ 탈출은 독립 edge이므로 다음 방문에서 실패 진입만 억제한다.

본 논문의 기여는 다음과 같다.

- **Task-conditioned active observation.** Goal과 주행 상태에 따라 current, directed 및 full sweep을 선택하여 한 장의 전방 영상이나 무조건적인 panorama보다 적절한 local route evidence를 얻는다.
- **Observation-bound causal transaction.** World point, robot-relative metric goal과 view-bound pixel을 구분해 transport하고 identity, freshness, ownership 및 terminal fence를 통과한 delayed decision만 원자적으로 handoff한다.

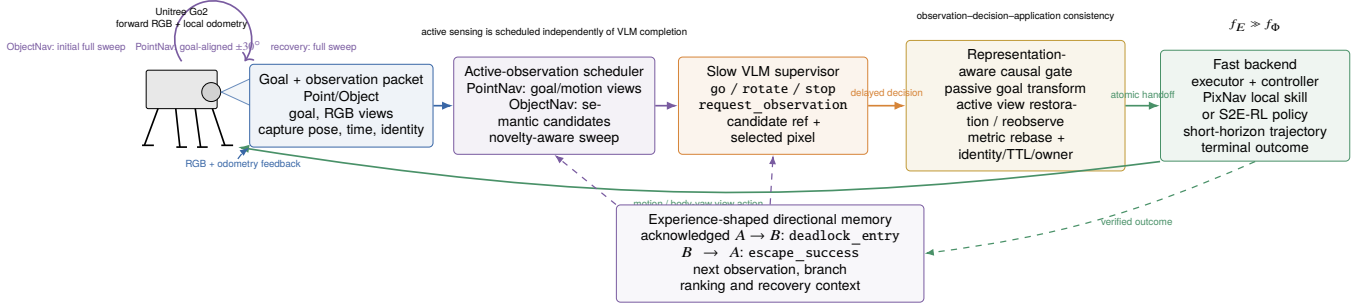


Fig. 1. ESCAPE-Nav의 system overview. Go2는 body-yaw view 뒤 admitted subgoal을 재개해 VLM inference와 주행을 중첩한다. Delayed output은 representation별 transport와 causal gate를 거치며, acknowledged outcome graph가 다음 observation과 branch/recovery를 형성한다.

• **Experience-shaped directional memory.** Acknowledged failed entry와 reverse escape를 독립 edge에 귀속하고 그 결과를 다음 관측·후보·복구 선택에 환류하여 deadlock 재진입과 oscillation을 억제한다.

평가는 HM3D indoor와 Habitat-GS outdoor의 paired PointNav에서 세 기전을 우선 분해한다. Localization corruption, NavBench-GS 재현, ObjectNav와 VLM backbone은 robustness와 generality의 보조 증거이며, Unitree Go2는 deployment validation으로 제한한다.

II. 관련 연구

Habitat와 HM3D는 photorealistic PointNav의 대표 평가 기반이며 [7]–[9], 이상적 RGB-D+GPS+Compass agent의 성능은 realistic sensing과 actuation noise에서 크게 저하된다 [1]–[3]. 본 연구는 localization을 추정하는 대신 RGB와 noisy relative goal/odometry를 사용하는 동일 조건에서 same-backend supervisor의 영향을 비교한다.

PixNav는 foundation model이 선택한 pixel을 RGB local skill에 연결하고 [10], S2E는 web-video pretraining과 simulation reinforcement learning을 outdoor navigation으로 확장했다 [11]. Active perception은 task에 맞게 sensor pose를 선택하며 [4], [5], ImagineNav는 미래 view를 이용한 VLM ObjectNav를 탐구했다 [6]. ESCAPE-Nav는 새로운 view generator를 학습하지 않고 fixed forward camera의 current/directed/full observation을 선택하며, 각 image를 capture pose와 immutable identity에 결합한다.

AsyncVLA와 SFCo-Nav는 slow foundation-model reasoning과 fast execution을 비동기로 결합하고 [12], [13], AsyncShield는 pose buffer와 kinematic mapping으로 delayed metric intent를 보상한다 [14]. 따라서 본 연구의 novelty는 asynchrony나 pose transform 자체가 아니다. ESCAPE-Nav는 world-, view- 및 robot-anchored goal을 구별하고, observation identity, owner와 terminal lifecycle을 검증한 뒤 원자적으로 적용하는 transaction을 제안한다. ReMemNav와 EvoMemNav는 episodic/visual-semantic memory로 navigation recovery를 돕는다 [15], [16]. 이에 비해 ESCAPE-Nav의 memory는 실제 handoff와 terminal acknowledgement가 확인된 directed transition만 갱신하여 failed entry와 reverse escape를 분리한다.

III. 방법

A. 문제 정의와 공통 구성

시간 t 의 robot body frame B 에 대한 episode-local planar pose를 $\mathbf{T}_{WB}(t) \in \text{SE}(2)$ 라 하자. W 는 episode 시작점의 local odometry frame이며, t 는 timestamp, 굵은 $\mathbf{T}$ 는 pose/transform에만 사용한다. 세 시점의 pose를

$$\mathbf{T}_k^O := \mathbf{T}_{WB}(t_k^O), \quad \mathbf{T}_k^C := \mathbf{T}_{WB}(t_k^C), \quad \mathbf{T}_k^A := \mathbf{T}_{WB}(t_k^A) \quad (1)$$

로 구분한다. 여기서 t_k^O, t_k^C, t_k^A 는 각각 k 번째 decision의 observation, S2E RGB context의 마지막 frame 및 application anchor의 timestamp다. 입력은 전방 monocular RGB I_t , PointNav relative goal $\tilde{\mathbf{g}}_t = (\tilde{\rho}_t, \tilde{\phi}_t)$ 또는 ObjectNav category c_g , local odometry이며, $\tilde{\rho}_t$ 와 $\tilde{\phi}_t$ 는 body-frame goal distance와 bearing이다. Persistent dense map은 사용하지 않고 sparse experience graph만 유지한다.

ESCAPE-Nav는 저주기 VLM supervisor Φ , 중간 주기 active-observation scheduler, 고주기 executor π_E 와 controller로 구성된다. k 번째 query는

$$\mathbf{q}_k = (\mathcal{O}_k, g_k, \mathcal{M}_k, \mathbf{T}_k^O, t_k^O, \mathbf{t}_k), \quad (2)$$

이다. $\mathcal{O}_k = \{(I_{k,j}, \delta\psi_{k,j}, t_{k,j})\}$ 는 image와 relative heading, g_k 는 canonical task goal이다. $\mathcal{M}_k$ 는 현재 place의 acknowledged directed outcome과 recovery state를 요약한 memory snapshot이며 raw image history가 아니다. $\mathbf{t}_k$ 는 episode, task, goal version과 observation sequence를 묶는 immutable identity다. VLM은

$$\mathbf{d}_k = (\alpha_k, r_k, \mathbf{z}_k, c_k, t_k^D, \mathbf{t}_k), \quad (3)$$

$$\alpha_k \in \{\text{go}, \text{rotate}, \text{request_observation}, \text{stop}\}, \quad (4)$$

를 출력한다. r_k 는 representation, $\mathbf{z}_k$ 는 candidate reference와 selected pixel, c_k 는 confidence/validity, t_k^D 는 completion time이다. Metric point는 backend가 calibration으로 계산하며 VLM은 자유 좌표나 low-level command를 생성하지 않는다. 이 유한 후보 선택은 hallucinated coordinate를 막고, 하나의 semantic choice를 서로 다른 backend 표현으로 재현 가능하게 전달한다.

a) *Prompted semantic supervision and structured contract.*: Canonical input은 task goal/state, capture metadata가 결합된 ordered RGB views, candidate set, memory snapshot과 action constraint다. Strict output은 Eq. (4)의 action, grounded view/candidate/pixel, confidence, time-to-live (TTL), 선택적 observation/memory proposal을 반환한다. Prompt는 PointNav

의 geometric stop과 ObjectNav의 visual confirmation을 구분하고 미제공 candidate/coordinate 생성을 금지한다. Backend는 identity, freshness, ownership, terminal state와 geometry를 재검증하며 memory는 acknowledged outcome만 commit한다.

State estimator는 GPS final goal을 relative PointGoal $(\tilde{\rho}_t, \tilde{\phi}_t)$ 로 변환하지만, 이는 따라야 할 doorway나 floor branch를 정하지 않는다. Candidate generator는 lower-image grid를 deterministic하게 샘플링하고 calibrated ray-ground-plane intersection으로 pixel을 capture-time metric point에 결합한 뒤 투영/RGB screening 실패를 제거한다. Φ 는 PointGoal, multi-view evidence와 memory로 coarse final goal을 한 local candidate로 refine한다. 동일 candidate를 PixNav에는 pixel, S2E에는 metric point로 전달한다. Grid와 cap은 Sec. IV-C에 공개하며, 선택점은 collision-free certificate가 아니라 locally executable hypothesis다.

B. 공통 active observation과 goal binding

Observation mode는 current, directed_view, directed_sweep, full_sweep이며, 그 집합과 scheduler 선택을 각각 $\mathcal{U}_O$ 와 $\mu_t \in \mathcal{U}_O$ 로 쓴다. Validated request를 $o_t^{\text{req}} \in \mathcal{U}_O \cup \{\emptyset\}$ 라 하면,

$$\mu_t = \begin{cases} \text{full_sweep}, & t = 0 \wedge \text{ObjectNav}, \\ \text{directed_sweep}, & t = 0 \wedge \text{PointNav}, \\ o_t^{\text{req}}, & o_t^{\text{req}} \in \mathcal{U}_O, \\ \text{current}, & \text{otherwise}, \end{cases} \quad (5)$$

로 선택한다. ObjectNav는 목표가 어느 방향에나 있을 수 있어 초기 full sweep을 사용하고, PointNav는 알려진 goal bearing 주변만 확인해 관측 비용을 줄인다. Candidate 부재 뿐 아니라 M_k 의 failed-edge recurrence, blocked/no-progress, oscillation 또는 place ambiguity가 확인될 때만 시야를 확장하며, 초기 PointNav candidate가 비면 한 번 full sweep한다. World-yaw novelty ledger는 같은 pose의 반복 관측을 막고, yaw bin이 소진되면 translation recovery 또는 다른 branch로 전환한다. Sweep 값은 Sec. IV-C에 명시한다.

Sync와 Async는 observation, novelty 및 grounding을 공유한다. Body-yaw capture가 잠시 motion을 점유한 뒤 Sync는 새 decision까지 정지하고, Async는 마지막 trajectory를 재개해 inference와 중첩한다. 각 bundle은 자체 timestamp와 capture pose를 갖는다.

두 실행 방식은 다음 representation contract를 공유하고, Async만 pose anchor 사이 transport를 추가한다.

World-anchored goal. 고정점 $^W\mathbf{g}$ 는 현재 body frame으로

$$\begin{bmatrix} B_t \mathbf{g} \\ 1 \end{bmatrix} = \mathbf{T}_{WB}(t)^{-1} \begin{bmatrix} ^W \mathbf{g} \\ 1 \end{bmatrix} \quad (6)$$

의 passive transform으로 갱신하며 Habitat PointGoal도 같은 robot-relative 의미를 갖는다.

View-anchored pixel. $\mathbf{u}_{k,j}$ 는 image, capture heading과 identity에 결합되므로 PixNav는 선택 view를 복원하거나 재관측하며 다른 view에 좌표를 복사하지 않는다.

Robot-relative metric goal. Candidate pixel은 ray-ground-plane intersection으로 $^{B_O} \mathbf{z}_k$ 에 결합되고 S2E가 short-horizon trajectory를 생성한다. 이는 view-bound pixel과 transport가 능한 metric point를 분리한다.

C. Synchronous reference system

Fig. 2(a)의 Sync는 O_k 획득 후 $\mathbf{d}_k$ 가 반환될 때까지 navigation을 보류한다. Contract 검증 뒤 PixNav에는 capture view/pixel, S2E에는 동일 pixel의 metric point를 전달하고, acknowledged outcome만 memory에 기록한다. Concurrent translation과 non-blocking handoff가 없어 흐름은 명확하지만 VLM latency만큼 wall time이 높고 duty cycle이 낮아진다. 따라서 Sync는 최종 방식이 아니라 공통 dataflow를 고정하고 Async에서 달라지는 시간 정렬만 분리해 설명하기 위한 reference다.

D. Causally aligned asynchronous extension

Fig. 2(b)의 Async는 Sync의 stage를 유지하면서 capture 후 기존 trajectory와 inference를 중첩하고, Eq. (1)의 세 anchor를 이용해 delayed decision을 실제 application boundary에서 교체한다.

1) *Concurrent execution and timestamped pose reconstruction:* Body-yaw capture interval의 집합을 $\mathcal{T}_O$ 라 하자. Observation control policy π_O 와 fast executor π_E 사이의 motion ownership은

$$\mathbf{u}_t = \begin{cases} \pi_O(\mu_t), & t \in \mathcal{T}_O, \\ \pi_E(I_{t-h:t}, \bar{\mathbf{z}}_{k^-}, \hat{\mathbf{x}}_{t-h:t}), & t \notin \mathcal{T}_O, \end{cases} \quad f_E \gg f_\Phi. \quad (7)$$

여기서 $\mathbf{u}_t$ 는 commanded control, π_O 는 bounded body-yaw policy, h 는 history horizon, k^- 와 $\bar{\mathbf{z}}_{k^-}$ 는 마지막 admission index와 transported subgoal, $\hat{\mathbf{x}}_{t-h:t}$ 는 estimated state history다. f_E 와 f_Φ 는 executor와 supervisor의 frequency이며 $f_E \gg f_\Phi$ 다. Capture 후 π_E 가 기존 subgoal을 재개한다. 비동기 query의 관측·context·적용 시각은 odometry sample과 일반적으로 일치하지 않으므로, 단순 latest-pose 조회는 이동 중 공간 오차를 만든다. 이에 $t^- < t < t^+$ 인 같은 frame의 $\mathbf{p}^- = (x^-, y^-, \theta^-)$ 와 $\mathbf{p}^+ = (x^+, y^+, \theta^+)$ 를 $\lambda = (t - t^-)/(t^+ - t^-)$ 로 보간한다.

$$\hat{\mathbf{p}}(t) = \begin{bmatrix} (1 - \lambda)x^- + \lambda x^+ \\ (1 - \lambda)y^- + \lambda y^+ \\ \mathcal{W}_\pi(\theta^- + \lambda(\mathcal{W}_\pi(\theta^+ - \theta^-))) \end{bmatrix} \quad (8)$$

로 translation은 선형, yaw는 shortest angle로 보간한다. $\mathcal{W}_\pi : \mathbb{R} \rightarrow (-\pi, \pi]$ 는 angle wrapping이다. Bracket 부재, frame 불일치 또는 과도한 간격에는 외삽하지 않으며 bound는 Sec. IV-C에 둔다. 이는 image/trajectory가 아니라 timestamped pose의 synchronization이다.

2) *Representation-aware delayed transport:* World goal은 Eq. (6)을 application pose에서 다시 계산한다. Pixel은 passive rebase할 수 없으므로 capture yaw $\psi_{k,j}^O$ 를

$$\delta\psi_k^{\text{view}} = \mathcal{W}_\pi(\psi_{k,j}^O - \psi_k^A) \quad (9)$$

로 복원해 quantize한다. Translation/scene change가 bound를 넘으면 pixel을 폐기하고 재관측한다.

S2E metric goal은 observation frame에서 RGB context frame으로 한 번 rebase한다. B_O, B_C, B_A 는 각 anchor의 body frame이다.

$$\begin{bmatrix} B_C \mathbf{z}_k \\ 1 \end{bmatrix} = (\mathbf{T}_k^C)^{-1} \mathbf{T}_k^O \begin{bmatrix} B_O \mathbf{z}_k \\ 1 \end{bmatrix}. \quad (10)$$

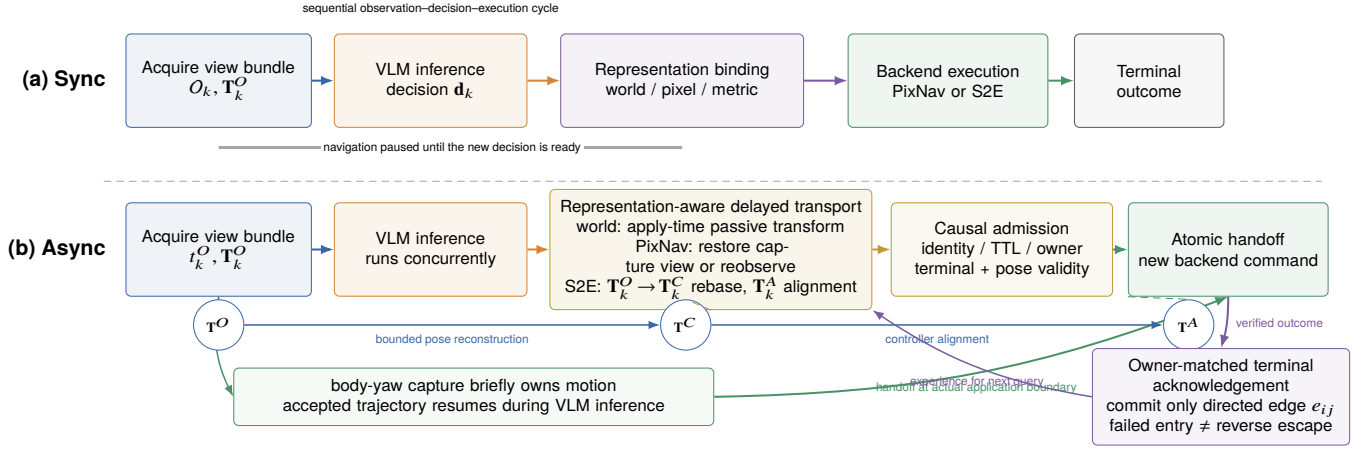


Fig. 2. Sync reference와 ESCAPE-Nav의 observation-bound Async transaction. Sync는 새 decision까지 정지하지만 Async는 trajectory와 VLM inference를 중첩한다. Delayed output은 pose reconstruction, representation별 transport와 causal admission 뒤 handoff된다. Owner-matched outcome만 graph를 갱신하고 다음 query에 환류되어 experience-shaped loop를 닫는다.

Trajectory frame은 T_k^C 로 고정하고 application state를

$$T_{BCBA,k} = (T_k^C)^{-1} T_k^A \quad (11)$$

로 표현하고 controller가 실제 application boundary에서 남은 short-horizon trajectory만 연결한다.

3) *Admission and non-blocking handoff*: Decision이 application time t_k^A 에 도착하면

$$\begin{aligned} \Gamma(d_k, t_k^A) = & \mathbb{I}[t_k = t(t_k^A)] \mathbb{I}[t_k^A - t_k^O \leq \tau_{\max}] \\ & \cdot \mathbb{I}[\omega_k^{\text{exp}} = \omega(t_k^A)] \mathbb{I}[b_T(t_k^A) = 0] \mathbb{I}[b_{P,k} = 1] \end{aligned} \quad (12)$$

$\Gamma = 1$ 만 admission한다. $\mathbb{I}[P]$ 는 indicator, $t(t_k^A)$ 는 active identity, $\tau_{\max}$ 는 TTL, ω_k^{exp} 와 $\omega(t_k^A)$ 는 expected/current owner다. b_T 는 terminal, $b_{P,k}$ 는 pose와 displacement validity flag다. Identity/goal 변경, expiration, owner mismatch, terminal 또는 invalid pose는 fail-closed하며, 이 gate는 physical safety가 아닌 causal admissibility를 보장한다.

rotate는 요청 yaw에서 observation-application 사이 이미 수행한 yaw를 뺀 wrapped residual만 실행한다. Admission 후 atomic handoff와 owner-matched acknowledgement를 사용한다. 따라서 Eq. (8)의 reconstruction, Eqs. (9)–(11)의 transport와 Eq. (12)가 causal binding을 보존한다.

E. Experience-shaped directional memory와 recovery

Experience는 raw image history나 parameter update가 아니라 owner-matched terminal acknowledgement로 확인된 directed action–outcome이다. 이를 sparse graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ 에 저장하며, node v_i 는 keyframe, semantic description, pose belief, visit count와 recovery state를 보존한다. Place recognition은 contribution이 아니다. Fixed prototype descriptor로 top- k 를 검색한 뒤 semantic/relative-pose consistency를 검증하고, 모호한 revisit는 duplicate node와 soft link로 유지한다. 따라서 claim은 승인된 revisit의 directional outcome attribution과 feedback에 한정된다.

Directed edge는

$$e_{ij} = (v_i, v_j, \Delta \hat{T}_{ij}, y_{ij}, n_{ij}^+, n_{ij}^-, \pi_{ij}) \quad (13)$$

이다. $\Delta \hat{T}_{ij}$ 는 relative transform, y_{ij} 는 success, blocked, deadlock_entry, escape_success 등의 outcome, n_{ij}^+, n_{ij}^- 는 acknowledged count다. Provenance π_{ij} 는 decision/command ID, pose, time, owner와 terminal evidence를 담는다. e_{ij} 와 e_{ji} 는 독립이며 graph는 handoff 후 acknowledged outcome만 갱신한다. 즉 causal alignment가 stale 또는 잘못 귀속된 결과로 experience가 오염되는 것을 막는다.

Recovery trigger D_r 는 blocked/collision evidence, K_s 회 연속 no-progress, K_o 회 no-progress와 heading reversal의 결합, 또는 recent graph recurrence 중 하나가 확인되면 활성화된다. 값은 Sec. IV-C에 공개하며 online trigger와 stored-edge penalty는 구분한다.

Acknowledged negative outcome set $\mathcal{Y}^-$ 는 deadlock_entry, deadlock_entry_candidate, blocked, risky를 포함한다. Candidate c 의 goal-bearing alignment를 $A_{\text{goal}}(c) \in [0, 1]$, negative-edge indicator를 $C_{\text{dead}}(c)$, recent recurrence의 capped penalty를 $C_{\text{osc}}(c)$, unvisited-view indicator를 $I_{\text{front}}(c)$ 라 하면

$$S_{\text{mem}}(c) = \lambda_a A_{\text{goal}}(c) - \lambda_d C_{\text{dead}}(c) - \{1 - C_{\text{dead}}(c)\} C_{\text{osc}}(c) + \lambda_f I_{\text{front}}(c). \quad (14)$$

Deadlock/oscillation penalty는 중복 적용하지 않고 frontier prior는 unvisited view에만 쓴다. Cap, recurrence rate와 weight는 Sec. IV-C에 둔다. Incident edge와 S_{mem} 를 요약한 $\mathcal{M}_k$ 는 Eq. (2)의 다음 query에 들어가고, failed-edge recurrence는 Eq. (5)의 validated request o_i^{req} 를 만든다. 후보가 없으면 관측을 넓히거나 recovery한다. Fig. 2의 verified outcome $\rightarrow$ graph mutation $\rightarrow$ next query가 experience-shaped loop이고, directed edge는 reverse escape와 다른 branch를 보존한다.

F. Backend contract와 구현 세부사항

Backend는 PixNav local skill 또는 S2E-RL policy이고, executor는 backend 호출, ownership, handoff와 terminal acknowledgement를 관리한다. Contract는 Eq. (4)의 action, representation metadata와 verified outcome으로 구성된다. 두 adapter는 같은 candidate identity를 공유하며 PixNav는 capture view/pixel, S2E는 결합된 metric point를 소비한다. 이하 ESCAPE-Nav에 PixNav와 S2E-RL을 연결한 instantiation

TABLE I
EVALUATION SETTINGS AND THEIR ROLE IN THE EVIDENCE HIERARCHY.

Setting	Task/backend	Scale	Purpose
HM3D	PointNav/PixNav	200 ep.	grounding, primary comparison
Habitat-GS	PointNav/S2E-RL	fixed set	outdoor, latency
HM3D recovery	PointNav/PixNav	40 ep.	deadlock, memory
HM3D	ObjectNav/PixNav	100 ep.	semantic-goal transfer
NavBench-GS	PointNav/S2E	26 scenes	evaluator calibration
Unitree Go2	PointNav	5P/method	deployment validation

을 각각 ESCAPE-PixNav와 ESCAPE-S2E로 표기한다. 별도 variant 표시가 없으면 active observation, causal transport와 directional memory를 모두 사용하는 full system이다. PixNav Native는 독립 baseline이고 ESCAPE-PixNav는 동일 skill에 ESCAPE-Nav supervisor, scheduler와 memory를 결합한다.

S2E-RL은 공개 S2E-BC를 초기화로 저자가 RL post-training한 backend다. Residual-Attention Module (RAM), proximal policy optimization (PPO), reward/curriculum의 재현 범위는 실험에 명시한다. ESCAPE-Nav결합 시 추가 gradient update가 없으므로 training-free는 integration에만 한정한다.

Unitree Go2 runtime은 고정 전방 RGB, Robot Operating System (ROS) [17] odometry와 body-yaw primitive를 사용한다. Contract의 right-turn-positive degree를 adapter 경계에서 ROS/controller의 counter-clockwise-positive radian으로 한 번 변환한다. Simulator pose를 clean oracle로 쓸 때는 명시하고 모든 robustness method에 같은 corruption trace를 적용한다.

IV. 실험 설계

평가는 세 research question (RQ)에 집중한다. **RQ1**: task-conditioned observation이 forward-only 또는 매 query마다 전방 camera로 360°를 모두 훑는 fixed sweep보다 적은 관측 비용으로 주행 progress를 개선하는가? **RQ2**: representation-aware transaction이 VLM latency에서 Sync의 stop-and-go cost와 naive Async의 stale application을 함께 줄이는가? **RQ3**: acknowledged action-outcome experience가 다음 관측·branch 선택을 바꿀 때 flat history나 place-level blacklist보다 failed-branch re-entry와 oscillation을 줄이는가? Primary evidence는 각 mechanism의 same-backend paired comparison이다. PixNav/S2E 적용, localization degradation, ObjectNav와 VLM backbone은 generality/robustness의 보조 증거이며, NavBench-GS와 Unitree Go2는 각각 evaluator calibration과 deployment validation으로 제한한다.

A. 환경과 평가 subset

HM3D PointNav는 validation split의 최소 20개 scene에서 shortest-path length와 geodesic/Euclidean ratio로 증화한 200 episode를 결과 확인 전에 고정한다 [9]. 계산 제약으로 100 개만 실행하면 leaderboard-scale 결과가 아닌 fixed subset으로 명시한다. Habitat-GS는 별도의 고정 outdoor subset을 사용한다 [18]. 두 환경에서 모든 방법은 동일 episode identifier, start/goal, action budget, VLM seed와 corruption trace를 공유하고, 동일 relative PointGoal, RGB-only observation 및 success criterion을 사용한다. Primary comparison은 같은 frozen backend의 Direct-goal 대 ESCAPE-Nav이며, outdoor adaptation이 없는 DD-PPO/VO는 reference로만 해석한다.

Random episode에서 드문 recovery event는 goal-aligned dead end, blocked branch와 symmetric/repeated corridor를 포함한 40개 controlled episode로 보완한다. 결과 확인 전에 simulator top-view geometry에서 deadlock의 trap/escape boundary와 oscillation의 recurrent cycle/exit branch를 표시하고, original goal, progress threshold와 hold horizon을 고정한다. 관측 variant가 선택한 local goal은 동일 frozen backend로 실제 실행하며, 실행 전후 hidden true-goal geodesic distance는 평가에만 사용하고 method 입력에는 제공하지 않는다.

ObjectNav는 동일 PixNav skill의 semantic-goal transfer만 확인한다 [19]. NavBench-GS는 원 논문의 26개 scene, embodiment, controller, timeout과 네 condition을 그대로 사용하며 Habitat-GS outdoor 결과와 합산하지 않는다 [11]. 본문에는 equal-weight macro-average를 제시하되 condition별 BC rerun을 result manifest에 공개하고, 네 condition 모두에서 일치할 때만 evaluator calibration을 주장한다. Go2는 하나의 고정 map에서 P 개 start-goal pair를 사전 고정하고 Direct-goal과 Full ESCAPE-Nav를 pair별 5회 실행한다. Intervention은 실패로 처리하며 Full-only recovery/dynamic-obstacle trial은 배치 범위만 확인한다. Pair별 RGB, trajectory와 event log는 video와 함께 보존한다.

B. 비교 방법

HM3D PointNav baselines.

- 1) *DD-PPO RGB*: RGB+GPS+Compass PointNav agent를 외부 learned-policy reference로 사용한다 [1]. Depth를 사용하는 DD-PPO result는 본 비교에서 제외한다.
- 2) *DD-PPO RGB under corruption* 및 *Robust VO PointNav*: test-time localization sensitivity와 noise-aware realistic policy를 비교한다 [2], [3]. Robust VO는 최종 평가 전에 단일 공개 method/checkpoint와 sensor input을 고정하며, 재현 가능한 checkpoint를 확보하지 못하면 일반명 baseline을 유지하지 않는다.
- 3) *Direct-goal PixNav*: current view의 executable candidate 중 PointGoal bearing error가 가장 작은 pixel을 선택하며, 후보가 없으면 goal 방향으로 deterministic rotate한다. *Direct-goal S2E-RL*: 현재 robot-relative PointGoal을 backend horizon에 맞게 제한해 metric goal로 전달한다. 두 방법 모두 VLM, active sweep 및 memory를 사용하지 않으며 tie-break와 horizon은 run manifest에 고정한다.
- 4) *ESCAPE-PixNav (IV)*: ESCAPE-PixNav와 동일한 supervisor contract, directional memory, async transport/admission 및 frozen PixNav backend를 유지하되, active acquisition만 제거하고 각 query의 최신 전방 RGB 한 장을 사용한다. Localization-degraded track에서도 다른 방법과 동일한 corrupted relative goal, odometry 및 RGB stress trace를 입력받는다. 이 variant는 candidate-grounding audit, ObjectNav transfer 및 active-observation ablation에서 사용하고, PointNav 주 결과는 동일 backend의 Direct-goal 대 full ESCAPE-Nav비교에 집중한다.
- 5) *ESCAPE-PixNav와 ESCAPE-S2E*: 동일 supervisor contract와 mechanism을 서로 다른 frozen backend에 적용한다. PixNav Native는 ObjectNav baseline이므로 이 비교와 Table III에서 제외한다.

HM3D ObjectNav baselines. *PixNav Native*는 원 protocol의 multi-view prompting과 pixel skill을 유지한다 [10].

TABLE II
KEY PARAMETERS. [TBD]는 평가 전 고정하고 전체 CONFIGURATION은 RUN MANIFEST에 공개한다.

항목	값
ObjectNav sweep/candidates	headings $\{0, \pm 60, \pm 120, -180\}^\circ$; directed $7 \times 2/14$, panorama $3 \times 2/4$
PointNav sweep/candidates	goal align $\{-30^\circ, 0^\circ, +30^\circ\}$; $\bar{u} \in \{.25, .375, .5, .625, .75\}$, $\bar{v} \in \{.84, .90\}$
RGB screening	view당 top-3, temperature 0
Pose interpolation bracket gap	0.25 s
Async gate/transport bounds	TTL/displacement, PixNav restoration/reobserve, S2E continuity: [TBD]
Deadlock evidence	$\epsilon_s = .08$ m, $K_s = K_o = 3$, turn 120° , return 45° , history 20
Memory ranking/retrieval	$\lambda_a = .55$, $\lambda_d = .90$, $c_{\max} = .5$, $\alpha_o = .25$, $\lambda_f = .1875$; edge top-8, revisit top-5
Place recognition	fixed 256-dimensional prototype descriptor, cosine top-5, $\tau_{pr} = .86$
Observation/recovery trigger	request 또는 blocked/no-progress/loop/empty candidate
Controlled recovery evaluation	$\delta_D, \delta_O, H_D, H_O$ 와 maximum recovery steps: [TBD]
Localization corruption	$\nu \in \{0, .25, .5, .75, 1\}$; goal/ego profile maxima from held-out calibration
Pose-stream delay levels	held-out Go2 median/95th percentile; stress value [TBD]
Metric thresholds	progress δ_p and translation duty ϵ_v : [TBD]

ESCAPE-PixNav는 같은 frozen PixNav backend에 ESCAPE의 task-conditioned observation, causal transport와 directional memory를 결합한다. 두 방법을 동일한 100 episode에서 평가하고 SR/SPL은 결과 본문에 직접 보고한다. Fig. 5는 동일 scene, start pose와 target category의 paired trajectory를 비교한다.

S2E reproduction. 공식 저장소가 제공하는 S2E-BC checkpoint를 동일 evaluator에서 먼저 재실행하여 protocol을 calibration한다. 이후 공개 BC를 초기화로 사용해 저자가 RL post-training한 *S2E-RL (ours)*를 평가한다. RAM 구조, trainable parameter, PPO objective, reward와 curriculum을 원 논문과 동일하게 재현한 범위는 본문의 실험 설정에 명시한다. 정확히 일치하지 않는 요소가 있으면 “independent reimplementation” 대신 “our RL-post-trained S2E”로 표기한다. ESCAPE-S2E결합 시 S2E-RL weight는 frozen이다.

C. 구현 및 재현 설정

Method의 수식은 mechanism과 acceptance condition만 정의하고, 결과와 acceptance를 직접 바꾸는 핵심 수치 parameter는 Table II에 공개한다. 나머지 camera/controller/backend 값도 최종 평가 전에 고정하여 prompt/schema hash와 함께 run manifest에 보존한다.

전체 framework와 Habitat runner는 Ubuntu 24.04 x86-64, Python 3.9.23, PyTorch 2.8.0/CUDA 12.8 및 Habitat 0.3.3 headless에서 구현하며, $2 \times$ NVIDIA RTX PRO 6000 Blackwell Server Edition GPU를 사용한다. VLM은 OpenAI-compatible application programming interface (API)로 독립 serving하여 simulator/navigation process와 분리하고, pose frame은 odom/base_link로 고정한다. Place retrieval은 grayscale thumbnail, RGB statistics와 histogram을 seeded fixed projection한 256-dimensional ℓ_2 -normalized prototype descriptor를 사용한다. 이 descriptor는 top-5 candidate만 제안하며 semantic/relative-pose verification 없이는 hard merge하지 않는다. 입력 image resolution, camera calibration, controller

primitive/frequency, context sampling, timeout, residual clamp 및 backend acceptance threshold는 공개 run manifest에 기록한다. 실험 중 값을 변경하면 별도 configuration으로 취급하며 같은 표의 결과를 혼합하지 않는다. 현재 smoke configuration의 success radius는 1.0 m이지만, 최종 Habitat/Habitat-GS 평가는 동일한 저장 trajectory에 0.2 m와 1.0 m success criterion을 각각 적용하여 재계산한다. Smoke run의 기존 1.0 m summary를 Table III에 직접 혼합하지 않는다.

D. Mechanism ablations

Exhaustive 조합 대신 다음 최소 ablation을 사전 고정한다. Table V은 동일 paired subset에서 구성요소 하나만 제거하고, latency sweep은 Fig. 3에 분리한다. 별도 명시 없으면 observation/memory는 clean pose와 native latency를 쓰며 async/pose block만 delay 또는 pose availability를 바꾼다.

Active observation. Async transport, memory와 frozen backend를 고정한 채 *Forward-only IV*, *Fixed 360° 6V* 및 ESCAPE의 *Adaptive* observation을 비교한다. Fixed 6V는 매 query마다 전방 RGB camera로 $\{0, 60, 120, 180, -120, -60\}^\circ$ 를 순차 관측하고 시작 heading으로 복귀한다. Adaptive는 task state에 따라 필요한 heading과 관측 횟수를 선택하며 별도 최대 장수를 강제하지 않는다. SR/SPL과 실제 images/query 및 sweep acquisition time을 함께 보고하여 더 많이 보는 효과와 선택적으로 보는 효과를 분리한다.

Causal async와 representation-aware transport. 모든 variant는 같은 active-observation policy, directional memory, VLM contract와 frozen backend를 사용하고 scheduler/transport만 바꾼다. Sync는 capture부터 decision handoff까지 기존 navigation을 보류한다. Async-direct는 기존 trajectory를 계속 실행하지만 delayed pixel, metric goal과 rotation을 observation-time 표현 그대로 적용하며 representation transport와 causal admission을 사용하지 않는다. 이 intentionally unaligned variant는 simulator에서만 실행한다. Async+geometric rebase는 timestamped pose reconstruction과 metric/rotation rebase를 사용하지만 view-bound pixel의 restoration/re-observation 및 identity, freshness, owner, terminal fence는 제거한다. ESCAPE-Nav async는 Eqs. (8)–(12)의 representation별 transport, admission 및 atomic handoff를 모두 사용한다. Table V의 transport block은 HM3D/PixNav의 공통 $L = 2$ s endpoint에서 SR/SPL, invalid/stale application, motion duty와 completion time을 함께 보고하여 Sync의 stop-and-go cost와 Async-direct의 stale-application cost를 분리한다. 이 표의 Async+pose only는 capture/apply pose는 복원하지만 pixel viewpoint restoration/re-observation과 causal fence를 제거한 PixNav용 partial variant다. Fig. 3는 complementary Habitat-GS/S2E-RL에서 Async+geometric rebase를 포함한 full latency curve를 제공하여 metric representation에서도 같은 경향을 확인한다. Apply-time heading error, query count와 application yield는 별도 diagnostic으로 기록한다. AsyncShield의 pose transformation 일부만 구현한 경우 이를 AsyncShield reproduction이라 부르지 않고 Async+geometric rebase로 명명한다 [14].

Pose synchronization. Pose interpolation의 기여는 동일 pose stream에서 bracketing interpolation을 사용하는 full variant와 query 시점의 latest-available pose sample만 사용하는 variant를 비교한다. 별도 pose-stream delay Δt_{pose} 는 held-out Go2 log의 median, 95th percentile 및 하나의 stress

level로 고정하고, 각 시점에는 $t - \Delta t_{\text{pose}}$ 까지 도착한 sample 만 노출한다. Table V은 primary 95th-percentile delay에서 capture/apply transform error, metric-goal error, invalid/stale application과 SR/SPL을 보고하며, median과 stress endpoint는 본문에 effect size와 CI로 병기한다. 최종 지연값은 평가 전에 Table II에 공개한다. Episode-level final goal은 수신 및 검증된 뒤 navigation clock을 시작하므로 초기 goal-message communication delay는 path-quality 실험에서 제외한다. 주행 중 갱신되는 localization delay는 이 pose-stream 실험이 담당한다.

VLM backbone portability. 기본 Qwen3-VL-32B-Instruct-AWQ와 두 개의 cross-size/family multi-image VLM을 동일 HM3D subset, ordered view bundle, prompt/schema 및 $L = 2$ s에서 비교한다. Model-specific prompt tuning 없이 SR/SPL과 schema/grounding validity를 확인하고, native latency와 application yield는 배치 비용으로 별도 기록한다. 세부 결과는 본문 한 문장과 공개 result manifest에 제공하며 primary table을 추가하지 않는다.

Experience-shaped memory. Controlled recovery subset에서 *No memory*, token-matched *flat history*, *failed-node blacklist*, *undirected edge*와 *full directional edge*를 비교한다. 관측/backend를 고정하고 과거 outcome의 feedback 방식만 바꾸므로 title claim을 직접 검증한다. 일반 episode의 SR/SPL과 함께 deadlock recovery rate (DRR), oscillation resolution rate (ORR) 및 average recovery steps (ARS)를 보고하며, 모든 variant에 동일한 controlled trial과 판정 규칙을 적용한다.

E. Matched goal/pose localization degradation

DD-PPO에만 noise를 주고 ESCAPE-Nav에는 clean PointGoal 또는 ground-truth apply pose를 남기면 불공정하다. 따라서 final-goal GPS uncertainty와 ego-pose/odometry uncertainty를 분리해 생성하되, 같은 episode-seed trace를 모든 방법에 제공한다. 두 source는 모두 robot-relative goal과 delayed-decision transport에 사용되는 spatial relation을 손상시키므로, 결합 조건을 goal/pose localization corruption이라 부른다. $\nu = 0$ 은 simulator와 metric implementation을 확인하는 oracle control이고, $\nu = .25$ 를 사전 고정한 low-corruption operating point로 둔다. Table III은 goal과 ego-pose corruption을 함께 적용한 이 조건의 strict-radius 절대 성능을 비교하고, Fig. 4는 동일 composite corruption을 $\nu \in \{0, .25, .5, .75, 1\}$ 로 증가시킨 clean-normalized SPL retention을 보여준다. 즉 Table III과 Fig. 4는 GPS-only와 pose-only 실험이 아니라 동일 protocol의 operating point와 severity curve이며, 두 source의 분리는 아래 diagnostic에서 수행한다.

$$\tilde{\mathbf{g}} = \mathbf{g} + \delta^g, \quad (15)$$

$$\tilde{\mathbf{x}}_t = \mathbf{x}_t + \mathbf{b}_t^x + \epsilon_t^x, \quad (16)$$

$$\mathbf{b}_t^x = \mathbf{b}_{t-1}^x + \eta_t^x, \quad (17)$$

$$\tilde{\mathbf{q}}_t = \mathbf{R}(\tilde{\psi}_t)^\top (\tilde{\mathbf{g}} - \tilde{\mathbf{p}}_t). \quad (18)$$

여기서 $\mathbf{g}$ 와 $\tilde{\mathbf{g}}$ 는 true/coarse final-goal coordinate, δ^g 는 episode 동안 고정한 goal-coordinate bias다. $\mathbf{x}_t = [\mathbf{p}_t^\top, \psi_t]^\top$ 와 $\tilde{\mathbf{x}}_t = [\tilde{\mathbf{p}}_t^\top, \tilde{\psi}_t]^\top$ 는 clean/corrupted ego pose, $\mathbf{b}_t^x$ 는 position/yaw 누적 bias다. ϵ_t^x 와 η_t^x 는 각각 zero-mean white-noise sample과 random-walk increment이며, $\tilde{\mathbf{q}}_t$ 는 모든 방법에 제공하는 noisy

robot-relative PointGoal이다. Severity ν 는 좌표에 직접 더하는 값이 아니라 goal과 ego profile

$$\theta_j(\nu) = (1 - \nu)\theta_{j,0} + \nu\theta_{j,\max}, \quad j \in \{g, x\}, \quad \nu \in [0, 1], \quad (19)$$

로 조절한다. θ_g 는 goal-bias scale, θ_x 는 position/yaw white-noise 및 random-walk 표준편차, dropout probability/duration과 sparse-jump rate/magnitude를 모은 벡터다. $\theta_{j,0} = \mathbf{0}$ 은 clean oracle, $\theta_{j,\max}$ 는 calibration으로 고정한 severe profile이다. Dropout 동안에는 마지막 ego-pose estimate를 유지하고, jump event에는 해당 severity의 position/yaw offset을 $\mathbf{b}_t^x$ 에 더한다. DD-PPO와 각 supervisor/direct-goal baseline은 같은 $\tilde{\mathbf{q}}_t$ 를 받고, odometry와 async rebase는 같은 $\tilde{\mathbf{x}}_t$ 를 사용한다. 따라서 robustness track의 noise는 RGB corruption, actuation noise, VLM inference latency 또는 message latency가 아니라 goal-coordinate 및 ego-localization의 *matched spatial corruption*이다. Success radius $r \in \{0.2, 1.0\}$ m와 noise severity $\nu \in \{0, .25, .5, .75, 1\}$ 에 대해

$$\text{SPL}_r(\nu) = \frac{1}{N} \sum_{i=1}^N S_i^{(r)}(\nu) \frac{\ell_i}{\max(p_i(\nu), \ell_i)}. \quad (20)$$

여기서 $S_i^{(r)}(\nu)$ 는 hidden true goal $\mathbf{g}$ 에 대한 final geodesic distance가 r 이내이면 1, 아니면 0인 indicator이고, ℓ_i 와 $p_i(\nu)$ 는 true start-goal shortest-path length와 실제 path length다. Noisy goal $\tilde{\mathbf{g}}$ 는 method 입력에만 사용하고 성공 판정에는 사용하지 않는다. 따라서 실패 episode도 평균의 분모 N 에 포함되며 SPL 기여는 0이다. Main table은 primary $r = .2$ m만 사용하고, $r = 1.0$ m 및 모든 severity의 exact SR/SPL은 공개 result manifest에 함께 제공한다.

Noise profile은 final test episode와 분리된 calibration log에서 고정한다. Reference goal/pose와 Go2 GPS-odometry의 goal-coordinate error, per-meter translation/yaw drift, update dropout duration, sparse jump magnitude와 event rate를 측정하고 $\theta_{g,\max}$ 와 $\theta_{x,\max}$ 의 각 값을 calibration distribution의 95th percentile로 정한다. 물리 calibration 범위를 넘는 추가 실험은 *stress*로 명시하고 realistic noise와 구분한다. Physical log가 충분하지 않으면 별도 pilot subset의 frozen direct-goal baseline에서 severe SPL retention이 floor에 도달하지 않는 범위를 선택하되, ESCAPE-Nav결과나 final test episode를 severity 선정에 사용하지 않는다. 각 nonzero severity에서 episode당 최소 세 corruption seed를 생성하고, 같은 episode-seed trace를 모든 방법에 재사용한다. 두 maximum profile의 수치, seed와 episode별 trace는 configuration manifest에 공개한다.

각 corruption factor의 full severity sweep을 모든 방법에 반복하지 않는다. 대신 두 환경의 $\nu = .5$ 에서 goal-only, ego-only 및 combined trace를 DD-PPO, Direct-goal과 동일-backend ESCAPE-Nav에 적용하여 coarse-goal refinement와 pose transport의 민감도를 분리한다. 이 source ablation의 strict SPL은 결과 본문 한 문장에 exact 값으로 병기하고 episode-level manifest도 공개한다. Main robustness claim은 Fig. 4의 matched composite curve로 판단한다. Translation-only, yaw-only, dropout/jump-only 및 RGB stress는 failure diagnostic으로 기록하되 main figure와 혼합하지 않는다. 초기 goal-message delay는 navigation clock 전에 종료하고, 주행

중 localization delay는 pose-synchronization ablation이 담당한다.

F. Latency protocol, 지표 및 통제

VLM delay는 $L \in \{0, 0.5, 1, 2, 4, 6\}$ s로 주입한다. 여기서 L 은 response-ready 시각에 더하는 controlled delivery delay이며, 예를 들어 $L = 2$ s는 모든 backbone의 응답을 정확히 2 s 늦게 전달하는 공통 조건이다. Primary sweep은 paired closed-loop rollout이다. 각 run에서 실제 생성된 query를 temperature zero로 한 번 decoding하여 response payload와 observation identity를 고정하고, payload를 바꾸지 않은 채 delivery timestamp에 L 만 더한다. Scheduler별 trajectory가 달라지면 이후 query도 달라질 수 있으므로 서로 다른 scheduler에 하나의 고정 observation trace를 강제로 재사용하지 않는다. 대신 동일 episode, initial state, simulator seed, model, prompt/schema와 decoding policy를 공유한다. 별도의 fixed-trace microbenchmark는 동일 recorded response의 stale rejection, pose error와 handoff event만 검증하며 navigation SPL 근거로 사용하지 않는다. Fig. 3는 closed-loop latency sensitivity를 보고한다. 이후 model/version과 server를 고정하고 strict schema-constrained decoding을 사용하는 live VLM subset으로 native end-to-end latency, query count, input images/episode, application yield와 useful applications/min을 검증한다. Prompt 및 schema version/hash를 각 run에 기록하며 mechanism ablation에서는 model과 prompt를 모두 고정한다. VLM-backbone ablation은 공통 prompt/schema를 유지하되 backbone만 교체하고 fixed-latency와 native-latency 결과를 분리한다. Burst delay, response drop과 out-of-order completion은 primary curve와 분리한 stress case로 보고한다.

Latency가 path quality뿐 아니라 completion efficiency에 미치는 영향을 실패 episode까지 포함해 비교한다. Strict 0.2 m success indicator를 S_i 라 하고 공통 wall-clock timeout을 $T_{\max}$ 라 할 때, episode i 의 집계 시간과 method m 의 normalized completion time은

$$T_i^\dagger = S_i \min(T_i, T_{\max}) + (1 - S_i)T_{\max},$$

$$\bar{T}_m(L) = \frac{1}{N} \sum_{i=1}^N \frac{T_{i,m}^\dagger(L)}{T_{\max}}. \quad (21)$$

으로 정의한다. 따라서 실패, collision termination 및 timeout은 모두 1의 normalized time cost를 가지며 조기 실패가 빠른 완료로 오인되지 않는다. Fig. 3(b)는 $\bar{T}_m(L)$ 와 paired bootstrap 95% CI를 보고하고, Table V의 transport block은 HM3D/PixNav $L = 2$ s에서 실제 second 단위 $T_i^\dagger$ 평균을 보고한다.

표준 metric은 success rate (SR), success weighted by path length (SPL), SoftSPL, final distance와 collision/m이다 [8]. SoftSPL은 Eq. (20)의 binary success $S_i^{(r)}$ 를 $\max(0, 1 - d_i/d_i^0)$ 로 대체한다. 여기서 d_i^0 와 d_i 는 hidden true goal까지의 시작 및 종료 geodesic distance이므로, strict-radius 실패도 실제 progress와 path efficiency에 따라 부분 점수를 얻는다. PointNav의 SR과 SPL은 strict 0.2 m와 near-goal 1.0 m success radius에서 모두 계산하며, 실패 episode도 평균에 포함한다. 0.2 m 결과를 primary criterion으로 해석하고 1.0 m 결과는 near-goal 도달과 terminal precision을 구분하는 보조 기준으로 사용한다.

운영 metric의 VLM latency는 request dispatch부터 response 수신 및 parse 완료까지의 end-to-end 시간을 completed request 전체에서 평균한 값이다. Duty는 navigation wall time 중 $|v| > \epsilon$ 인 translation 시간의 비율이고, Yield는 completed VLM response 중 Eq. (12)를 통과해 backend에 실제 적용된 비율이다. 따라서 Duty는 Sync의 stop-and-go 비율을, Yield는 async 결과가 stale/invalid rejection으로 낭비되지 않는지를 측정한다. Controller frequency는 novelty metric이 아니므로 main result에서 제외하고 target/achieved value를 implementation log에만 공개한다. 실로봇의 Intv.는 human safety intervention 횟수/run, Rec.는 triggered recovery 중 original-goal pursuit를 재개한 비율이며 trigger가 없는 method에는 보고하지 않는다.

Active-observation의 Prog.는 선택된 local goal을 frozen backend가 실제 실행하기 직전과 종료 직후의 hidden true-goal geodesic distance 차가 δ_p 이상인 decision 비율이다. 모든 selected-candidate decision을 분모에 포함하고, causal gate에서 거절되거나 backend outcome이 없는 decision의 progress는 0으로 처리한다. Img./q는 VLM query당 실제 입력 RGB 수, Obs.는 sweep 시작부터 heading 복귀 또는 중단까지 걸린 acquisition time의 평균이다. 따라서 별도 hidden-navmesh counterfactual rollout 없이 실제 주행 성능, 관측량과 시간 비용을 함께 비교한다. S-valid는 completed response 중 schema parse와 required-field 검증을 통과한 비율, G-valid는 그 schema-valid response 중 제공된 candidate identifier와 representation-specific grounding이 일치한 비율이다.

Controlled recovery trial 집합을 deadlock $\mathcal{D}$ 와 oscillation $\mathcal{O}$ 로 사전 구분하고 trial i 의 해결 여부를 $r_i \in \{0, 1\}$ 로 둔다. Deadlock은 trap region을 벗어나 original goal의 geodesic distance를 δ_D 이상 줄이거나 다음 waypoint에 도달하고, 이후 H_D step 동안 failed branch에 재진입하지 않을 때 해결된다. Oscillation은 recurrent cycle을 벗어나 cycle 밖의 새 branch/node로 진행하고 original goal distance를 δ_O 이상 줄인 뒤, H_O step 동안 같은 cycle을 반복하지 않을 때 해결된다. Trigger가 발생하지 않아 탈출하지 못한 경우도 전체 trial의 실패로 남긴다. 해결 시점의 step을 s_i , 사전 고정된 최대 recovery step을 M_i 라 하면

$$\text{DRR} = \frac{\sum_{i \in \mathcal{D}} r_i}{|\mathcal{D}|}, \quad \text{ORR} = \frac{\sum_{i \in \mathcal{O}} r_i}{|\mathcal{O}|},$$

$$\text{ARS} = \frac{\sum_{i \in \mathcal{D} \cup \mathcal{O}} [r_i s_i + (1 - r_i) M_i]}{|\mathcal{D}| + |\mathcal{O}|}. \quad (22)$$

여기서 DRR과 ORR은 각각 Deadlock Recovery Rate와 Oscillation Resolution Rate이고, ARS는 Average Recovery Steps이다. 미해결 trial을 M_i 로 charge해 성공한 쉬운 trial만 평균하는 편향을 막는다. 표에는 비율과 함께 해결 수/전체 수의 raw count를 병기하며, annotation geometry와 $(\delta_D, \delta_O, H_D, H_O, M_i)$ 는 결과 확인 전에 고정한다. 모든 비교는 paired episode로 수행한다. 결과 확인 전 primary endpoint를 RQ1의 strict-radius SPL 및 progress rate, RQ2의 strict-radius SPL, normalized completion time 및 invalid/stale application, RQ3의 DRR, ORR와 ARS로 고정한다. 이 primary method difference에만 paired bootstrap 95% confidence interval (CI)을 사용하고 SR 차이는 McNemar test로 확인한다. Fig. 3는 curve uncertainty를 CI band로 표시한다. Table

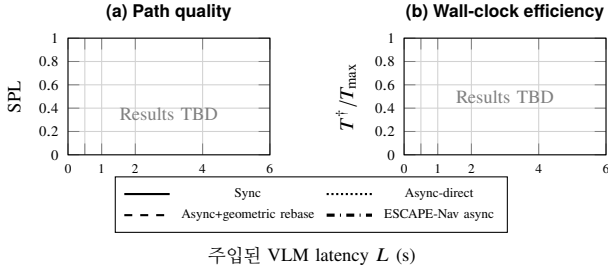


Fig. 3. VLM-latency sensitivity. 각 condition의 query는 한 번만 deterministic decoding하고 전달 시작에만 L 을 더한다. Condition은 episode/seed를 공유하지만 observation trace는 공유하지 않는다. (a)는 SPL, (b)는 Eq. (21)의 normalized time과 95% CI이며, Table V은 HM3D/PixNav의 $L = 2$ s diagnostic이다.

TABLE III

POINTNAV UNDER MATCHED GOAL/EGO-POSE CORRUPTION AT $\nu = .25$. SUCCESS USES THE HIDDEN TRUE GOAL AND 0.2 M RADIUS; S-SPL IS SOFTSPL. PRIMARY COMPARISONS ARE SAME-BACKEND DIRECT-GOAL VERSUS ESCAPE-NAV. CLEAN ($\nu = 0$) DIRECT/ESCAPE-NAV SR-SPL: HM3D/PixNav [TBD]–[TBD]/[TBD]–[TBD]; HABITAT-GS/S2E-RL [TBD]–[TBD]/[TBD]–[TBD].

Method	Backend	HM3D			Habitat-GS		
		SR	SPL	S-SPL	SR	SPL	S-SPL
DD-PPO RGB	policy	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Robust VO	policy	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Direct-goal	PixNav	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
ESCAPE-Nav	PixNav	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Direct-goal	S2E-RL	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
ESCAPE-Nav	S2E-RL	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

의 secondary diagnostic과 실로봇 deployment metric은 point estimate와 raw denominator만 보고하여 표를 단순화한다. 각 RQ 내부의 사전 지정 pairwise comparison에는 Holm correction을 적용하며, 실패 run, excluded simulator error, completed/applied/rejected query와 controlled-trial denominator를 method별로 공개한다.

V. 결과 보고 및 논의

A. 핵심 기전 결과

세 same-backend comparison이 핵심이다. Adaptive observation은 Fixed 360° 6V보다 높은 progress/SPL 또는 같은 성능에서 낮은 image/acquisition cost를 보여야 한다. Full transaction은 Sync의 time/duty와 Async-direct의 invalid/stale application 및 SPL을 함께 개선해야 한다. Directional edge가 flat history, node blacklist와 undirected edge보다 DRR/ORR을 높이고 ARS를 줄이면서 일반 SR/SPL을 보존할 때만 experience-shaped claim을 지지하며, 실패한 claim은 보조 결과로 대체하지 않는다.

Table V은 mechanism별 핵심 지표와 memory의 일반 SR/SPL을 함께 보고한다. Fixed/adaptive의 views/query와 yaw travel은 manifest에, DRR/ORR은 해결 수/전체 수와 함께 본문에 기록하며, ARS는 Eq. (22)처럼 미해결 trial을 고정 horizon으로 포함한다.

B. Generality와 calibration

두 추가 VLM backbone의 fixed-latency SR/SPL과 schema/grounding validity는 본문에, native latency/yield는 manifest에 보고한다. 동일 100 HM3D ObjectNav episode

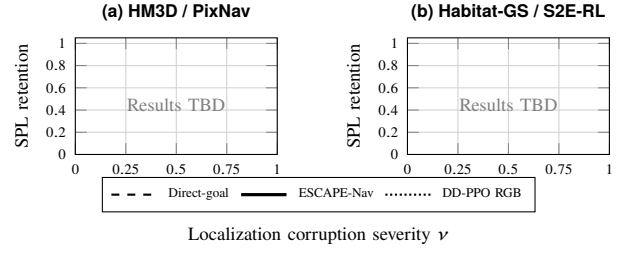


Fig. 4. Matched goal-coordinate/ego-pose corruption. Curves show strict SPL normalized by each method's $\nu = 0$ value; Table III reports absolute $\nu = .25$ performance. Same-backend Direct-goal versus ESCAPE-Nav tests causal supervision; DD-PPO is a reference and source-specific diagnostics are separate.

TABLE IV
NAV BENCH-GS FOUR-CONDITION
MACRO-AVERAGE. RC/CT는 S2E PROTOCOL;
†: REPORTED, ‡: RERUN.

Method	SR ↑	RC ↑	CT ↓
S2E-BC†	0.59	0.76	1.23
S2E-Full†	0.68	0.83	0.84
S2E-BC rerun‡	[TBD]	[TBD]	[TBD]
Direct S2E-RL‡	[TBD]	[TBD]	[TBD]
ESCAPE-S2E‡	[TBD]	[TBD]	[TBD]

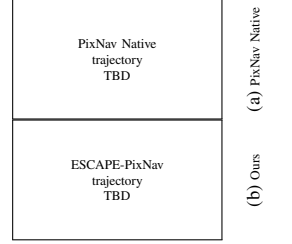


Fig. 5. Comparison between PixNav Native and ESCAPE-PixNav (ours) on matched HM3D ObjectNav trajectories.

에서 PixNav Native와 ESCAPE-PixNav의 SR-SPL은 각각 [TBD]–[TBD]와 [TBD]–[TBD]이며, Fig. 5는 paired trajectory를 보인다. Table IV의 †/‡는 reported/rerun이고, 불완전한 재현은 “our RL-post-trained S2E”로 한정하여 Table III의 same-backend 효과와 분리한다.

C. 정성 분석과 실로봇 배치

Fig. 6는 HM3D의 Direct-goal/active selection과 Habitat-GS의 Async-direct/full transaction을 비교하며, deadlock/oscillation 근거는 Table V의 DRR/ORR/ARS와 raw count로 한정한다. Fig. 1의 Go2 setup에서 고정한 P 개 pair의 결과/runtime은 Table VI, trajectory는 video/log에 공개하고 추가 recovery/dynamic-obstacle trial은 deployment 증거로만 해석한다.

D. 한계

ESCAPE-Nav는 global localization이나 formal collision safety를 보장하지 않고, 큰 pose drift는 rebase와 graph association을 손상시킨다. RGB-only 관측은 낮은 장애물, 불명확한 경계와 non-planar terrain에 취약하며 body-yaw에는 회전 비용이 든다. Ground-plane projection은 calibration/planarity에 의존하고, raw pixel과 sparse graph에는 각각 큰 scene change 뒤 재사용과 global optimization의 한계가 있다. 이는 depth-aware grounding, obstacle validation과 transport uncertainty의 동기다. 또한 relative PointGoal을 계속 받는 Habitat 평가는 완전한 GNSS-denied 탐사보다 쉬우므로 실제 극지·재난 성능으로 일반화할 수 없다.

VI. 결론

ESCAPE-Nav는 단일 전방 카메라와 느린 VLM navigation을 observation-bound causal transaction으로 정식

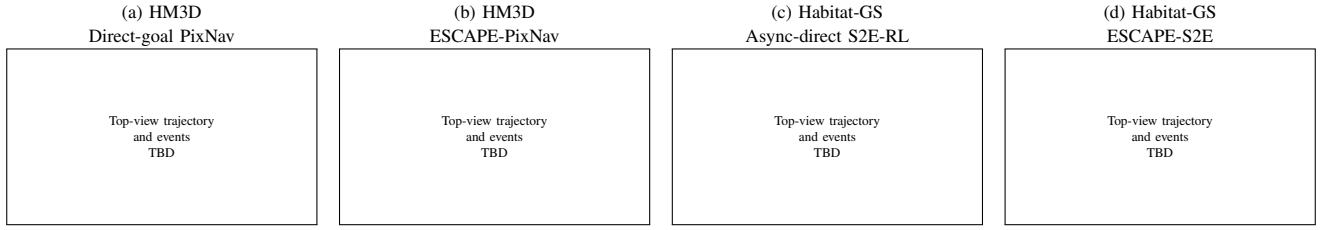


Fig. 6. Paired trajectories. (a)–(b) compare goal-bearing and active local-route selection; (c)–(d) compare unaligned delayed application and the full transaction. Each domain pair shares map extent and marks start, goal, trajectory, application/rejection events and terminal pose.

TABLE V

HM3D/ESCAPE-PixNAV MECHANISM ABLATIONS. INV.: INVALID/STALE APPLICATION; IMG./Q: INPUT IMAGES PER QUERY; OBS.: ACQUISITION TIME INCLUDING ABORTED SWEEPS.

Asynchronous transport and causal admission						
Variant	SR ↑	SPL ↑	Inv. ↓	Duty ↑	Time ↓	
Sync	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	
Async-direct	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	
Async+pose only	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	
ESCAPE-Nav async	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	
Pose synchronization within full async (held-out p95 delay)						
Variant	SR ↑	SPL ↑	TF err. ↓	Goal err. ↓	Inv. ↓	
Latest sample	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	
Interpolation	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	
Active observation policy						
Variant	SR ↑	SPL ↑	Prog. ↑	Img./q ↓	Obs. ↓	
Forward-only 1V	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	
Fixed 360° 6V	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	
ESCAPE adaptive	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	
Experience-shaped directional memory and controlled recovery						
Memory	SR ↑	SPL ↑	DRR ↑	ORR ↑	ARS ↓	
No memory	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	
Flat history	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	
Failed-node blacklist	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	
Undirected edge	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	
Directional edge	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	

TABLE VI

Go2 PAIRED NAVIGATION IN ONE FIXED MAP (5P TRIALS/METHOD). TIME: $T^\dagger$; INTV.: INTERVENTIONS/RUN; REC.: SUCCESSFUL/TRIGGERED RECOVERY; LAT.: MEAN VLM LATENCY.

Method	SR ↑	Intv. ↓	Time ↓	Rec. ↑	Lat. (s) ↓	Duty ↑	Yield ↑
Direct-goal	[TBD]	[TBD]	[TBD]	–	–	[TBD]	–
ESCAPE-Nav	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

화한다. Active observation은 local-route evidence를 얻고 representation-aware transport는 delayed decision을 정렬한다. 신뢰할 수 있게 귀속한 directed outcome이 다음 관측 · candidate · recovery를 바꾸는 페루프가 *experience-shaped*의 의미다. HM3D/Habitat-GS PointNav를 primary, transfer와 Go2를 secondary evidence로 평가하며 depth-aware grounding/transport는 future work다.

REFERENCES

- [1] E. Wijmans, A. Kadian, A. Morcos, S. Lee, I. Essa, D. Parikh, M. Savva, and D. Batra, “DD-PPO: Learning near-perfect pointgoal navigators from 2.5 billion frames,” in *International Conference on Learning Representations*, 2020.
- [2] X. Zhao, H. Agrawal, D. Batra, and A. G. Schwing, “The surprising effectiveness of visual odometry techniques for embodied pointgoal navigation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 16 127–16 136.
- [3] R. Partsey, E. Wijmans, N. Yokoyama, O. Doboşevych, D. Batra, and O. Maksymets, “Is mapping necessary for realistic pointgoal navigation?” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 17 232–17 241.
- [4] R. Bajcsy, “Active perception,” *Proceedings of the IEEE*, vol. 76, no. 8, pp. 996–1005, 1988.
- [5] S. Chen, Y. Li, and N. M. Kwok, “Active vision in robotic systems: A survey of recent developments,” *The International Journal of Robotics Research*, vol. 30, no. 11, pp. 1343–1377, 2011.
- [6] X. Zhao, W. Cai, L. Tang, and T. Wang, “ImagineNav: Prompting vision-language models as embodied navigator through scene imagination,” *arXiv preprint arXiv:2410.09874*, 2024.
- [7] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, D. Parikh, and D. Batra, “Habitat: A platform for embodied ai research,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 9339–9347.
- [8] P. Anderson, A. Chang, D. S. Chaplot, A. Dosovitskiy, S. Gupta, V. Koltun, J. Kosecka, J. Malik, R. Mottaghi, M. Savva, and A. Zamir, “On evaluation of embodied navigation agents,” *arXiv preprint arXiv:1807.06757*, 2018.
- [9] S. K. Ramakrishnan, A. Gokaslan, E. Wijmans, O. Maksymets, A. Clegg, J. Turner, E. Undersander, W. Galuba, A. Westbury, A. Chang, M. Savva, Y. Zhao, and D. Batra, “Habitat-matterport 3d dataset (HM3D): 1000 large-scale 3d environments for embodied ai,” in *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
- [10] W. Cai, S. Huang, G. Cheng, Y. Long, P. Gao, C. Sun, and H. Dong, “Bridging zero-shot object navigation and foundation models through pixel-guided navigation skill,” *arXiv preprint arXiv:2309.10309*, 2023.
- [11] H. He, Y. Ma, B. Squicciarini, W. Wu, and B. Zhou, “From seeing to experiencing: Scaling navigation foundation models with reinforcement learning,” in *International Conference on Learning Representations*, 2026, arXiv:2507.22028v2.
- [12] N. Hirose, C. Glossop, D. Shah, and S. Levine, “AsyncVLA: An asynchronous vla for fast and robust navigation on the edge,” *arXiv preprint arXiv:2602.13476*, 2026.
- [13] C. Xiong, L. Wei, X. Hu, K. Ma, Z. Xia, Z. Jiang, Z. Sun, and L. Pei, “SFCo-Nav: Efficient zero-shot visual language navigation via collaboration of slow llm and fast attributed graph alignment,” *arXiv preprint arXiv:2603.01477*, 2026.
- [14] K. Yang, Z. Chu, Y. Guo, Z. Wang, S. Xie, Y. Shen, X. Wu, X. Li, and M. Xu, “AsyncShield: A plug-and-play edge adapter for asynchronous cloud-based VLA navigation,” *arXiv preprint arXiv:2604.24086*, 2026.
- [15] F. Wu, W. Zuo, W. Yang, J. Xiao, Y. Liu, and X. Zeng, “ReMemNav: A rethinking and memory-augmented framework for zero-shot object navigation,” *arXiv preprint arXiv:2603.26788*, 2026.
- [16] Z. Ge, X. Jia, C. Wu, Y. Zhou, Z. Wu, and Y.-G. Jiang, “EvoMemNav: Efficient self-evolving fine-grained memory for zero-shot embodied navigation,” *arXiv preprint arXiv:2606.03509*, 2026.
- [17] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, “ROS: An open-source robot operating system,” in *ICRA Workshop on Open Source Software*, 2009.
- [18] Z. Xia, J. Xu, C. Cui, Y. Yu, J. Zhang, Q. Yan, T. Ni, J. Chen, X. Zhou, H. Bao, R. Hu, and S. Peng, “Habitat-GS: A high-fidelity navigation simulator with dynamic gaussian splatting,” *arXiv preprint arXiv:2604.12626*, 2026.
- [19] D. Batra, A. Gokaslan, A. Kembhavi, O. Maksymets, R. Mottaghi, M. Savva, A. Toshev, and E. Wijmans, “Objectnav revisited: On evaluation of embodied agents navigating to objects,” *arXiv preprint arXiv:2006.13171*, 2020.